Project Report: Personal Expense Tracker App

1. Introduction & Project Overview

The **Personal Tracker App** is a comprehensive desktop application engineered to help individuals monitor, log, and organize their daily metrics, tasks, and personal data efficiently. Built using Java, this project serves as a practical demonstration of core software engineering principles, robust data management, and user interface design.

2. New Concepts Learned in This Project

Developing this application introduced several foundational and intermediate programming concepts:

- **Graphical User Interface (GUI):** Moving beyond console-based programs, we learned how to build interactive desktop interfaces that respond dynamically to user actions.
- **Abstract Window Toolkit (AWT) & Swing:** We explored Java's AWT and Swing libraries to design

visual components such as windows, buttons, text fields, panels, and tables.

- **CSV File Handling:** We learned how to read from and write to Comma-Separated Values (CSV) files, enabling lightweight, human-readable file persistence without needing a heavy database setup.

3. What Did We Use?

- **Programming Language:** Java (Standard Edition)
- **UI Framework:** Java AWT & Swing for rendering windows, layout managers (e.g., FlowLayout, BorderLayout, GridLayout), and handling user event listeners.
- **Data Storage Format:** CSV (Comma-Separated Values) files for structured data serialization and text-based persistence.
- **Development Environment:** Integrated Development Environment (IDE) like IntelliJ IDEA or Eclipse for code compilation, debugging, and project management.

4. How We Did Things in the Code (Implementation Details)

The codebase is organized modularly to separate user interaction, core business logic, and file input/output operations.

- **UI Layout & Event Handling:** We initialized a main application frame (`JFrame`) and partitioned it using panels (`JPanel`). Components like `JTextField`, `JButton`, and `JTable` were added to capture user inputs (such as tracking entries, dates, and categories). Action listeners (`ActionListener`) were implemented to listen for button clicks, trigger validation checks, and update the UI state.
- **Data Flow:** When a user submits an entry through the GUI, the application instantiates corresponding model objects, validates the attributes, appends the data to an in-memory collection (like an `ArrayList`), and subsequently serializes the collection into the persistent CSV file.

5. Object-Oriented Programming (OOP) Pillars

The application architecture strictly adheres to the four pillars of Object-Oriented Programming to ensure clean, maintainable, and scalable code:

- **Encapsulation:** Data fields (such as tracker entry names, dates, values, and IDs) are kept `private` inside classes. Controlled access is provided via `public` getter and setter methods to protect the internal state of objects.
- **Inheritance:** Common attributes and behaviors across different types of trackable items were abstracted into a parent/base class, allowing specialized tracker classes to inherit shared functionality and reduce code duplication.
- **Polymorphism:** Method overriding was utilized to handle specific display and formatting logic differently depending on the subclass type, allowing uniform handling of diverse tracker data through common interfaces or superclass references.
- **Abstraction:** Abstract classes and interfaces were used to define strict contracts for data loaders, savers, and UI components, hiding complex implementation details from the high-level application logic.

6. Data Persistence and Error Mitigation

To ensure user data is safely stored and the application remains stable under unexpected conditions, we implemented robust mechanisms for data persistence and error handling:

- **Data Persistence (CSV IO):** Using Java's `FileWriter`, `BufferedWriter`, `Scanner`, and `BufferedReader`, the application writes application states directly to local CSV files upon saving and reads them back into memory upon startup. This ensures that user records persist across application restarts.
- **Error Mitigation & Exception Handling:**
  - **Try-Catch Blocks:** Implemented around file input/output (I/O) operations to gracefully catch `IOException` or `FileNotFoundException` if files are missing or corrupted.
  - **Input Validation:** Built-in checks prevent application crashes caused by empty fields, incorrect date formats, or invalid numerical inputs, displaying descriptive error messages or fallback states to the user instead.

7. Results and Outcomes

- **Functional Desktop Application:** Successfully delivered a fully working desktop tracker application featuring a responsive graphical interface and local file storage.
- **Structured Data Management:** Users can seamlessly input, view, and save their tracked data without data loss between sessions.
- **Clean Code Structure:** Applied rigorous OOP design patterns, resulting in modular code that is easy to debug, test, and expand with new features in the future.

8. Practical Life Applications

The Personal Tracker App can be adapted for various real-world scenarios to boost personal productivity and organization:

- **Habit & Routine Tracking:** Monitoring daily routines, fitness goals, or study hours.
- **Expense & Budget Management:** Keeping a lightweight log of daily expenditures and categories without needing complex banking software.
- **Task & Project Logging:** Tracking milestones, to-do lists, and task completion rates for personal or academic projects.

Group No.: 7

Team Members:

- **Farhan Zabir Alif** – 2522270642
- **Ashikul Hoque Abrar** – 2524870642
- **Hasibuzzaaman** – 2514222642
- **Ridia Afrin** – 2222344042